UNIVERSITY OF TEHRAN

Electrical and Computer Engineering Department

CAD

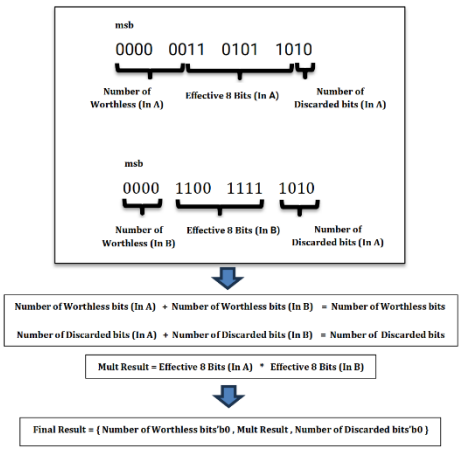
CA1

Name:

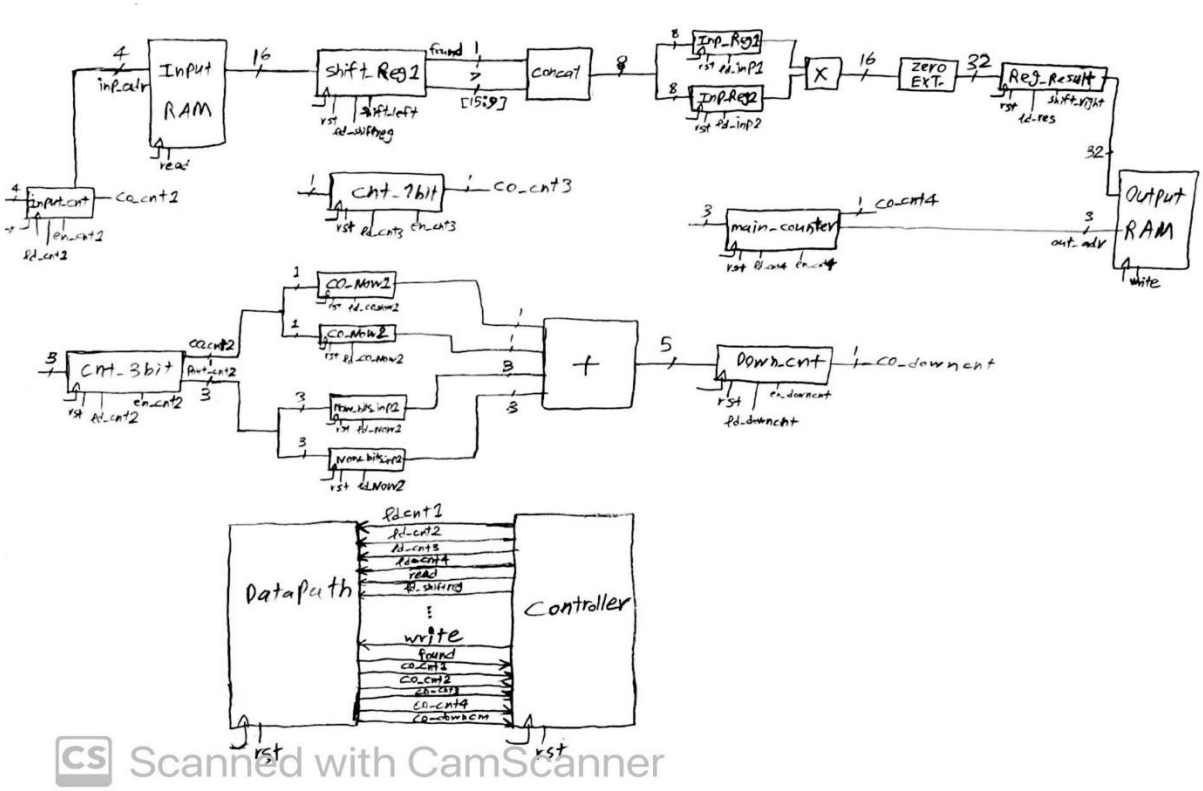
Mahdi Naieni 810101536

Amin Aghakasiri 810101381

Overview of project:



Data path:



Scanned with CamScanner

```verilog
timescale 1ns/1ns
module datapath(clk, rst, ld_cnt1, ld_cnt2, ld_cnt3, ld_cnt4, read, ld_shiftreg, ld_res, shift_left, shift_right, en_cnt1,
          ld_co_inp1, ld_co_inp2, en_cnt2, en_cnt3, en_cnt4,ld_inp1, ld_inp2, ld_now1, ld_now2, ld_downcnt, en_downcnt, write,
          found, co_cnt1, co_cnt2, co_cnt3, co_cnt4, co_downcnt);

    input clk, rst, ld_cnt1, ld_cnt2, ld_cnt3, ld_cnt4, read, ld_shiftreg, ld_res, shift_left, shift_right, en_cnt1 ,ld_co_inp1, ld_co_inp2,
          en_cnt2, en_cnt3, en_cnt4,ld_inp1, ld_inp2, ld_now1, ld_now2, ld_downcnt, en_downcnt, write;
    output found, co_cnt1, co_cnt2, co_cnt3, co_cnt4, co_downcnt;

    wire [4:0] add_out;
    wire [3:0] pout_cnt1, temp;
    wire pout_cnt3, shift_left_out, co_now1_out, co_now2_out;
    wire [2:0] pout_cnt2, pout_cnt4, now1_out, now2_out;
    wire [15:0] data, data_shift_left, mult_out;
    wire [7:0] concat_out, inp1_out, inp2_out;
    wire [31:0] zero_ext_out, shift_right_out;


    counter #(4) cnt_1(clk, rst, ld_cnt1, en_cnt1, 4'b0000, co_cnt1, pout_cnt1);
    counter #(3) cnt_2(clk, rst, ld_cnt2, en_cnt2, 3'b000, co_cnt2, pout_cnt2);
    counter #(1) cnt_3(clk, rst, ld_cnt3, en_cnt3, 1'b0, co_cnt3, pout_cnt3);
    counter #(3) cnt_4(clk, rst, ld_cnt4, en_cnt4, 3'b000, co_cnt4, pout_cnt4);

    input_ram inp_ram(clk, pout_cnt1, read, data);
    shift_left_reg shift_l (clk, rst, ld_shiftreg, shift_left, data, shift_left_out, data_shift_left);

    assign found = shift_left_out;
    concat conc(shift_left_out, data_shift_left[15:8], concat_out);

    register #(8) inp_1(clk, rst, ld_inp1, concat_out, inp1_out);
    register #(8) inp_2(clk, rst, ld_inp2, concat_out, inp2_out);

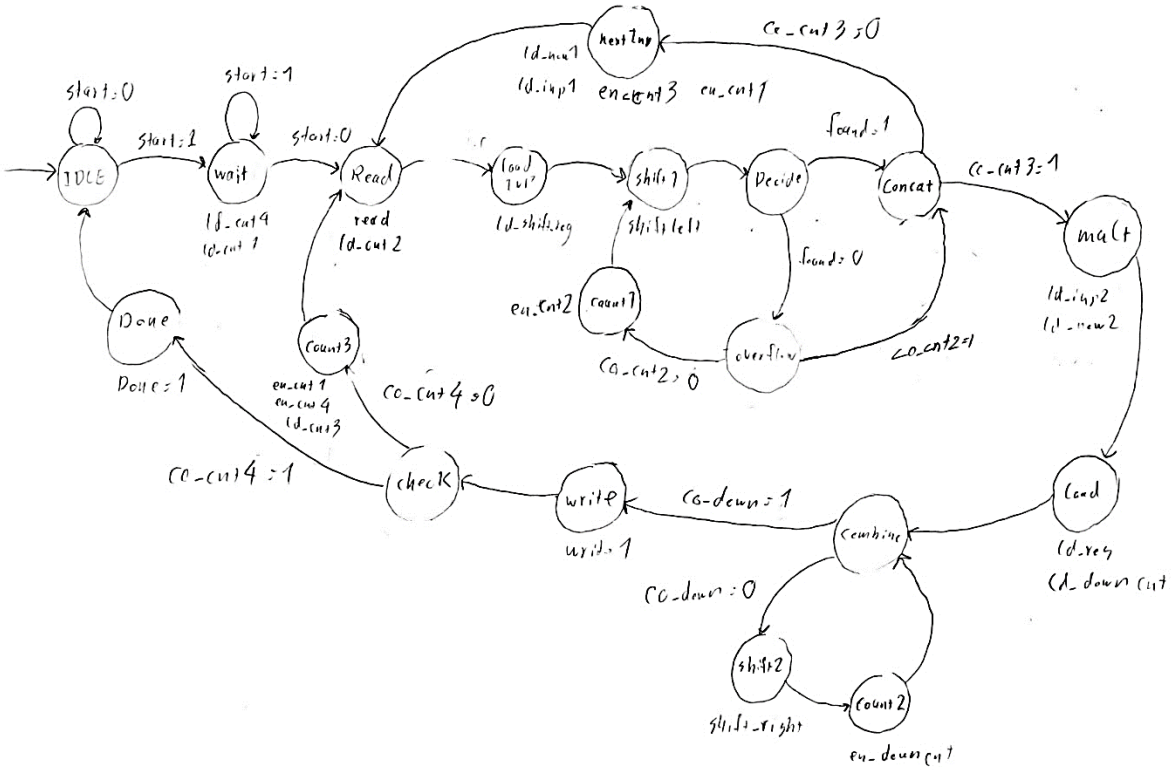
    mult multiplier(inp1_out, inp2_out, mult_out);
    zero_extend zero_ext(mult_out, zero_ext_out);
    shift_right_reg shift_r(clk, rst, ld_res, shift_right, zero_ext_out, shift_right_out);
    output_ram out_ram(clk, write, pout_cnt4, shift_right_out);


    register #(3) now_1(clk, rst, ld_now1, pout_cnt2, now1_out);
    register #(3) now_2(clk, rst, ld_now2, pout_cnt2, now2_out);
    register #(1) coo_now_1(clk, rst, ld_co_inp1, co_cnt2, co_now1_out);
    register #(1) coo_now_2(clk, rst, ld_co_inp2, co_cnt2, co_now2_out);

    adder adding(now1_out, now2_out, {2'b0, co_now1_out}, {2'b0, co_now2_out}, add_out);
    down_counter down_cnt(clk, rst, ld_downcnt, en_downcnt, add_out, co_downcnt, temp);


endmodule
```

Controller:



Controller Code:

```
`define Count1 5'b00111
`define Correction 5'b01000
`define Concat 5'b01001
`define NextInp 5'b01010
`define Mult 5'b01011
`define Load 5'b01100
`define Combine 5'b01101
`define Shift2 5'b01110
`define Count2 5'b01111
`define Write 5'b10000
`define Check 5'b10001
`define Count3 5'b10010
`define Done 5'b10011


module controller (clk, rst, start, found, co_cnt1, co_cnt2, co_cnt3, co_cnt4, co_downcnt,
                   ld_cnt1, ld_cnt2, ld_cnt3, ld_cnt4, read, ld_shiftreg, ld_res, shift_left, shift_right, en_cnt1,
                   en_cnt2, en_cnt3, en_cnt4,ld_inp1, ld_inp2, ld_now1, ld_now2, ld_downcnt, en_downcnt, write, ld_co_inp1, ld_co_inp2, done);

    input clk, rst, start, found, co_cnt1, co_cnt2, co_cnt3, co_cnt4, co_downcnt;

    output ld_cnt1, ld_cnt2, ld_cnt3, ld_cnt4, read, ld_shiftreg, ld_res, shift_left, shift_right, en_cnt1,
           en_cnt2, en_cnt3, en_cnt4,ld_inp1, ld_inp2, ld_now1, ld_now2, ld_downcnt, en_downcnt, write, ld_co_inp1, ld_co_inp2, done;

    reg ld_cnt1, ld_cnt2, ld_cnt3, ld_cnt4, read, ld_shiftreg, ld_res, shift_left, shift_right, en_cnt1,
        en_cnt2, en_cnt3, en_cnt4,ld_inp1, ld_inp2, ld_now1, ld_now2, ld_downcnt, en_downcnt, write,ld_co_inp1, ld_co_inp2, done;

    reg [4:0] ps, ns;
    always @(posedge clk) begin
        if(rst)
            ps <= `Idle;
        else
            ps <= ns;
    end

    always @(ps, start, found, co_cnt1, co_cnt2, co_cnt3, co_cnt4, co_downcnt) begin
        case(ps)
            `Idle: ns = start ? `Wait : `Idle;
            `Wait: ns = start ? `Wait : `Read;
            `Read: ns = `LoadInp;
            `LoadInp: ns = `Shift1;
            `Shift1: ns = `Decide;
            `Decide: ns = found ? `Concat : `Overflow;
            `Overflow: ns = co_cnt2 ? `Correction : `Count1;
            `Count1: ns = `Shift1;
            `Correction: ns = `Concat;
            `Concat: ns = co_cnt3 ? `Mult : `NextInp;
            `NextInp: ns = `Read;
            `Mult: ns = `Load;
            `Load: ns = `Combine;
            `Combine: ns = co_downcnt ? `Write : `Shift2;
            `Shift2: ns = `Count2;
            `Count2: ns = `Combine;
            `Write: ns = `Check;
            `Check: ns = co_cnt4 ? `Done : `Count3;
            `Count3: ns = `Read;
            `Done: ns = `Idle;
            default: ns = `Idle;
        endcase
    end

    always @(ps) begin
        {ld_cnt1, ld_cnt2, ld_cnt3, ld_cnt4, read, ld_shiftreg, ld_res, shift_left, shift_right, en_cnt1} = 10'b0;
        {en_cnt2, en_cnt3, en_cnt4, ld_inp1, ld_inp2, ld_now1, ld_now2, ld_downcnt, en_downcnt, write, done} = 11'b0;
        {ld_co_inp1, ld_co_inp2} = 2'b0;
        case(ps)
            `Idle: ;
            `Wait: {ld_cnt1, ld_cnt4} = 2'b11;
            `Read: {read, ld_cnt2} = 2'b11;
            `LoadInp: {ld_shiftreg} = 1'b1;
            `Shift1: {shift_left} = 1'b1;
            `Decide: ;
            `Overflow: ;
            `Count1: {en_cnt2} = 1'b1;
            `Correction:;
            `Concat: ;
            `NextInp: {ld_now1, ld_inp1, en_cnt1, en_cnt3, ld_co_inp1} = {4'b1111, ~found};
            `Mult: {ld_inp2, ld_now2, ld_co_inp2} = {2'b11, ~found};
            `Load: {ld_res, ld_downcnt} = 2'b11;
            `Combine:  ;
            `Shift2: {shift_right} = 1'b1;
            `Count2: {en_downcnt} = 1'b1;
            `Write: {write} = 1'b1;
            `Check:  ;
            `Count3: {en_cnt1, en_cnt4, ld_cnt3} = 3'b111;
            `Done: {done} = 1'b1;
        endcase
    end

endmodule
```

Sample Testbench:

